

TECHNICAL FILE ANALYSIS GUIDE

P-FUND Backend Code & Logic Line-by-Line Documentation

This document provides an exhaustive codebase explanation, outlining the logical operations, security checks, database transactions, helper routes, and model interactions inside the P-FUND APIs system.

Designed as a reference manual for developers onboarding or performing audits on the backend structure.

Author: Laravel Development Team

Target Framework: Laravel 11.x / Sanctum

Generation Date: May 25, 2026

Status: Active Production Code

1. API Routing Layer

api.php

Location: routes/api.php

Overview:

This file specifies the URI structures for all REST endpoints. It enforces Laravel Sanctum authentication (auth:sanctum) and utilizes custom role middlewares (role:creator, role:admin, role:vetter, role:sponsor) to lock down access routes.

Method: Route::prefix('v1/auth')

Groups all public authentication utilities under the 'v1/auth' namespace prefix. This includes endpoint mappings for user signup, logins, password reset email links, and resets. Sub-routes requiring authenticated sessions are grouped under 'auth:sanctum' middleware.

```
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
```

Method: Route::middleware('auth:sanctum')->prefix('v1')

Defines the authenticated core API group. Shared profile viewing (GET /me), global chat postings (GET/POST /chat), notification lookups, and document downloaders are declared here so any authenticated user can hit them.

```
Route::get('/me', [ProfileController::class, 'show']);
Route::get('/chat', [ChatController::class, 'index']);
```

Method: Role-Based Middleware Isolation

Protects resource portals. By wrapping route groups with custom 'role:creator', 'role:admin', 'role:vetter', or 'role:sponsor' middlewares, the routing system automatically returns a 403 response if the logged-in user's role does not match.

```
Route::middleware('role:creator')->prefix('projects')->group(...);
Route::middleware('role:vetter')->prefix('vetter')->group(...);
```

2. Middleware & Response Architecture

EnsureRole.php

Location: app/Http/Middleware/EnsureRole.php

Method: handle(Request \$request, Closure \$next, string ...\$roles)

Intercepts incoming requests to examine the user's role. It normalizes comparison by converting roles to lowercase. It includes specific wildcard logic for Vettors: if the endpoint requires 'vettor', any sub-role starting with 'vettor' (e.g. 'vettor_1', 'vettor_2', 'vettor_3') is granted entry. Aborts with a 403 JSON payload on mismatch.

```
foreach (array_map('strtolower', $roles) as $required) {  
    if ($userRole === $required) { $hasAccess = true; break; }  
    if ($required === 'vettor' && str_starts_with($userRole, 'vettor')) { ... }  
}
```

ForceJsonResponse.php

Location: app/Http/Middleware/ForceJsonResponse.php

Method: handle(Request \$request, Closure \$next)

Injected in the global middleware stack. It dynamically overwrites the incoming 'Accept' header to 'application/json'. This is crucial because it instructs Laravel's core router and exceptions handler to return validation or database errors in JSON format rather than executing standard web redirects to a login or welcome screen.

```
$request->headers->set('Accept', 'application/json');  
return $next($request);
```

ApiResponse.php

Location: app/Traits/ApiResponse.php

Method: success() & error() Methods

A reusable trait implemented across all controllers to maintain unified REST responses. success() formats payload fields and returns status codes (default 200). error() encapsulates failure descriptions and returns custom error structures and details (default 400).

```
protected function success(mixed $data = null, string $message = 'Success', int $status = 200)  
protected function error(string $message = 'Something went wrong', int $status = 400, mixed $errors = null)
```

3. Authentication Controller

AuthController.php

Location: app/Http/Controllers/Api/V1/AuthController.php

Method: register(Request \$request)

Performs registration validation including custom attributes (nationality, specialties, education). Wraps operations in a database transaction block (DB::beginTransaction) to ensure user creations are rolled back if system logs fail. On success, creates a Sanctum token, generates an EmailVerificationCode (6-digit OTP), dispatches a VerifyEmailNotification email, and logs the signup action.

```
$user = User::create($validated);
$token = $user->createToken('auth_token')->plainTextToken;
$otp = EmailVerificationCode::generateFor($user);
$user->notify(new VerifyEmailNotification($otp->code));
```

Method: verifyEmail(Request \$request)

Verifies user emails against 6-digit OTP codes. First checks if already verified. Locates verification models matching the active user ID and submitted code. If matching code exists, evaluates expiration time. On success, marks user as email_verified_at, logs activity, deletes verification code record, and returns a verified state.

```
$verification = EmailVerificationCode::where('user_id', $user->id)->where('code',
$request->code)->first();
$user->markEmailAsVerified();
$verification->delete();
```

Method: login(Request \$request)

Authenticates users by validating credentials using Web guard validate rules. Prevents login if credentials fail. Compares user's role with requested portal role (enforcing that Creators login to the Creator portal, Sponsors to the Sponsor portal, etc.). Accounts for vetting hierarchies (allowing sub-roles like vetter_1 to login as a general 'vetter'). Blocks logins for accounts marked as suspended (is_active = false) and generates a fresh token.

```
if (!Auth::guard('web')->validate($request->only('email', 'password'))) { ... }
if (!$user->is_active) { return $this->error('Your account has been suspended...', 403); }
```

Method: logout(Request \$request) / forgotPassword() / resetPassword()

logout() logs the session exit, deletes the current Sanctum token, and returns a success response. forgotPassword() leverages Laravel's native Password broker to construct and dispatch password reset links. resetPassword() validates the reset token, updates the database, and clears active Sanctum tokens.

```
$user->currentAccessToken()->delete();
Password::reset($request->only(...), function (User $user, string $password) { ... });
```

4. Profile Management Controller

ProfileController.php

Location: app/Http/Controllers/Api/V1/ProfileController.php

Method: show(Request \$request)

Retrieves the active user object using `loadCount('projects')` to aggregate statistics. Constructs the public HTTP URL for the user's avatar asset using Laravel's Storage facade and returns a structured profile details payload.

```
$user = $request->user()->loadCount('projects');  
$avatarUrl = $user->avatar ? Storage::disk('public')->url($user->avatar) : null;
```

Method: uploadAvatar(Request \$request)

Handles profile picture upload updates. Restricts uploads to JPEG/PNG/WebP formats with a 2MB cap. Ensures older avatar images are physically deleted from storage disks (`Storage::disk('public')->delete()`) to prevent disk space leaks. Saves the new file to an 'avatars' folder, updates the User database path column, and records the upload activity.

```
if ($user->avatar) { Storage::disk('public')->delete($user->avatar); }  
$path = $request->file('avatar')->store('avatars', 'public');  
$user->update(['avatar' => $path]);
```

Method: update(Request \$request)

Allows updates to profile details like name, phone, nationality, address, specialties, and education. Processes partial payload validations, updates the db, logs profile edits, and returns fresh user states.

```
$validated = $request->validate([ 'name' => ['sometimes', 'string', 'max:255'], ... ]);  
$user->update($validated);
```

Method: changePassword(Request \$request)

Enforces secure password updates. Verifies current password matches using `Hash::check()`. On validation success, updates password columns. To maintain active account security, it revokes all other active device login sessions/tokens, preserving only the current connection token.

```
if (!Hash::check($request->current_password, $user->password)) { return $this->error(...); }  
$user->tokens()->where('id', '!=', $request->user()->currentAccessToken()->id)->delete();
```

5. Creator & Project Management Controller

ProjectController.php

Location: app/Http/Controllers/Api/V1/ProjectController.php

Method: index(Request \$request) & show(Request \$request, Project \$project)

index() fetches projects owned by the logged-in creator, supports filtering by status query parameters, eager-loads attachments, and paginates responses. show() retrieves a specific project, checking ownership constraints before returning the project along with its milestones and deliverables.

```
$query = Project::forUser($request->user()->id)->with('documents')->latest();  
if ($project->user_id !== $request->user()->id) { return $this->error('Project not found', 404); }
```

Method: store(StoreProjectRequest \$request)

Enables project application uploads. Starts database transactions. Generates a project record linked to the creator. Iterates through uploaded file attachments, saving physical copies under sub-folders keyed to the project ID, and creates project document records matching details like file name, mime type, size, and paths.

```
DB::beginTransaction();  
$project = $request->user()->projects()->create($request->only(...));  
$path = $file->store('projects/' . $project->id . '/documents', 'public');
```

Method: update(UpdateProjectRequest \$request, Project \$project) & destroy()

update() validates that changes are only permitted while a project is in 'draft', 'rejected', or 'needs_update' statuses. If rejected, updates status back to 'draft'. destroy() allows creators to wipe draft applications. Iterates through records to physically delete document attachments from disks before removing the project record.

```
if (!$project->isDraft() && $project->status !== 'rejected' && ...) { return $this->error(...); }  
Storage::disk('public')->delete($document->file_path);
```

Method: submit(Request \$request, Project \$project)

Submits draft applications for official review. Checks if the project status is 'needs_update' to flag it as a resubmission, changing the state to 'vetting' instead of 'submitted'. Records activity, publishes in-app notification alerts for the creator, and sends an automated project status notification email.

```
$isResubmission = ($project->status === 'needs_update');  
$newStatus = $isResubmission ? 'vetting' : 'submitted';  
$project->update(['status' => $newStatus, 'submitted_at' => now()]);
```

6. Vetter Review Controllers

VetterProjectController.php

Location: app/Http/Controllers/Api/V1/VetterProjectController.php

Method: index(Request \$request) / show() / stats()

Allows Vettors to examine submitted projects. stats() returns counts of projects currently in the vetting queue or requiring creator action. index() queries projects with status 'submitted' or 'vetting' to populate their work queue. show() displays project profiles with documents and deliverables.

```
$queue = Project::whereIn('status', ['submitted', 'vetting'])->with('user')->paginate(...);
```

Method: requireUpdate(Request \$request, Project \$project)

Enables Vettors to request edits from Creators. Validates feedback payloads, updates project status to 'needs_update', logs the request, and dispatches in-app notification alerts to the Creator specifying what revisions are needed.

```
$project->update(['status' => 'needs_update']);
```

```
Notification::create([ 'user_id' => $project->user_id, 'type' => 'project_needs_update', ... ]);
```

Method: reject(Request \$request, Project \$project) & updateMilestone()

reject() updates project status to 'rejected' if reviews fail. updateMilestone() allows Vettors to update specific project milestone states to 'completed' or 'pending' as requirements are met.

```
$project->update(['status' => 'rejected']);
```

```
$milestone->update(['status' => $request->status]);
```

VetterDeliverableController.php

Location: app/Http/Controllers/Api/V1/VetterDeliverableController.php

Method: index() / show() / review(Request \$request, ProjectDeliverable \$deliverable)

Enables Vettors to review milestone deliverables uploaded by Creators. review() validates reviews (approves or requests edits), modifies deliverable status columns, and updates milestones to complete once approved.

```
$request->validate([ 'status' => ['required', 'in:approved,needs_correction'], 'feedback' => ... ]);
```

```
$deliverable->update(['status' => $request->status, 'feedback' => $request->feedback]);
```

7. Sponsor & Admin Management Controllers

SponsorProjectController.php

Location: app/Http/Controllers/Api/V1/SponsorProjectController.php

Method: dashboardStats(Request \$request) & fund(Request \$request, Project \$project)

dashboardStats() aggregates metrics for the sponsor's dashboard, counting active investments and total committed funds. fund() creates a SponsorCommitment record, updating project statuses and recording the transaction in activity logs.

```
$committed = SponsorCommitment::where('sponsor_id', $user->id)->sum('amount');
SponsorCommitment::create([ 'project_id' => $project->id, 'sponsor_id' => $user->id, 'amount' => ... ]);
```

AdminProjectController.php

Location: app/Http/Controllers/Api/V1/AdminProjectController.php

Method: review(Request \$request, Project \$project)

Allows Admins to review project applications, changing statuses to 'approved' or 'rejected'. Generates milestone templates for approved projects to prepare them for funding.

```
$project->update(['status' => 'approved', 'reviewed_at' => now()]);
// Generates milestones based on project scope
$project->milestones()->createMany([ ... ]);
```

AdminUserController.php

Location: app/Http/Controllers/Api/V1/AdminUserController.php

Method: store(Request \$request) & suspend(Request \$request, User \$user)

store() allows Admins to create new accounts (e.g. provisioning new Vетters or Admins). suspend() toggles the user's active status. Toggling 'is_active' to false blocks login access, logging the security event.

```
$user = User::create($validated);
$user->update(['is_active' => !$user->is_active]);
ActivityLog::create([ 'action' => $user->is_active ? 'USER_REINSTATED' : 'USER_SUSPENDED', ... ]);
```

8. Data Models & Eloquent Schema Relationships

User.php

Location: app/Models/User.php

Eloquent Setup & Helpers:

Inherits Laravel's Authenticatable class. Uses Sanctum's 'HasApiTokens' trait to manage authentication tokens. Defines fillable properties and casts user password inputs. Implements model helper methods to check roles (isAdmin, isCreator, isVetter, isSponsor) and includes an appended 'friendly_role' attribute for descriptive UI displays.

Project.php

Location: app/Models/Project.php

Attributes & Scopes:

Represents project campaigns. Defines constants for statuses ('draft', 'submitted', 'vetting', 'approved', 'rejected', 'needs_update'). Establishes relationships: hasMany documents, hasMany milestones, hasMany deliverables, and belongsTo creator (User). Includes query scopes like forUser() and byStatus().

Milestone.php & ProjectDeliverable.php

Location: app/Models/Milestone.php & ProjectDeliverable.php

Milestone-Deliverable Relationship:

Milestone maps out funding stages. It belongsTo a Project and hasMany ProjectDeliverable submissions. ProjectDeliverable tracks creator files submitted as evidence for a milestone, storing file names, paths, statuses ('pending', 'approved', 'needs_correction'), and feedback text.

EmailVerificationCode.php

Location: app/Models/EmailVerificationCode.php

OTP Generation Logic:

Handles verification codes. Defines a generateFor(User \$user) static method that deletes any active verification code records for the user, generates a new random 6-digit string, and saves it with a 15-minute expiration timestamp. Includes isExpired() checks.